

**Convert
PASSWORD...**



genSalt()



- This function in bcrypt is used to generate a salt. The first argument specifies the cost factor, which determines how computationally expensive the hash function will be to calculate.
- The higher the cost factor, the more iterations of the hashing algorithm will be performed, making it more computationally expensive for an attacker to crack the password. The cost factor is usually a number between 10 and 12.



Salt

A salt is a random data that is used as an additional input to a one-way function (such as a cryptographic hash function) that "hashes" data, typically passwords.

Salting is a technique used to add complexity to the hashing process, making it more secure.



// Requiring module

```
const bcrypt = require('bcrypt');
```

```
const password = 'pass123';
```

```
var hashedPassword="";
```

// Encryption of the string password

```
bcrypt.genSalt(10, function (err, Salt) {
```

```
    // The bcrypt is used for encrypting password.
```

```
    bcrypt.hash(password, Salt, function (err, hash) {
```

```
        if (err) return console.log('Cannot encrypt');
```

```
        hashedPassword = hash;
```

```
        console.log("convert password"+hash);
```



```
bcrypt.compare(password, hashedPassword,  
  async function (err, isMatch) {  
    // Comparing the original password to encrypted password  
    if (isMatch) {  
      console.log('Encrypted password is: ', password);  
      console.log('Decrypted password is: ', hashedPassword);  
    }  
    if (!isMatch) {  
      // If password doesn't match the following  
      // message will be sent  
      console.log(hashedPassword + ' is not encryption of ' + password);  
    }  
  })  
})
```